DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

24CSC51 - MACHINE LEARNING

REPORT

PREDICTIVE MAINTENANCE FOR HYDRAULIC PRESS AND HYDRAULIC TEST-RIG SYSTEMS

AKASH A

24CSR012

ARAVINDHAN S

24CSR024

BALAMURUGAN K

24CSR033

DESHAANTH M

24CSR048

PROBLEM STATEMENT

Hydraulic presses, injection units and power packs rely on four critical components: the cooler, the valve, the pump and the hydraulic accumulator. Each of these degrades gradually and independently of the others. The degradation is difficult to see from the outside, yet it shows up clearly in the pressure, volume-flow, temperature, vibration, motor-power and efficiency signals that are recorded during every working cycle of the machine. A hydraulic test-rig dataset records 2,205 working cycles of 60 seconds each, captured by 17 sensors sampled at 100 Hz, 10 Hz and 1 Hz, together with the known condition of all four components for every cycle.

When a component fails unexpectedly, the press stops, production is lost and the failure frequently damages other parts of the circuit. A worn cooler lets the oil overheat, a leaking pump overloads the motor, a lagging valve disturbs the pressure profile and a failing accumulator causes pressure shocks. Inspecting each component manually at fixed intervals is costly, and it either replaces healthy parts too early or detects the fault too late. Moreover, the raw sensor data contains hundreds of thousands of readings per cycle, which makes manual interpretation practically impossible.

Therefore, an effective data-driven approach is required to convert the multi-sensor cycle data into compact descriptive features and to identify the condition of every component automatically. This project aims to study how the sensor signals change across the degradation levels of each component, to engineer per-cycle statistical features from all 17 sensor channels, and to build classical machine learning models that classify the degree of degradation of the cooler, valve, pump and accumulator. The system also compares the single-target approach (one model per component) with a multi-output approach (one model predicting all four components), and analyses which sensor features influence each prediction the most. By supporting component-wise condition monitoring and informed preventive maintenance planning, the proposed approach aims to reduce unplanned press downtime, avoid secondary damage and lower maintenance cost.

ABSTRACT

Condition monitoring of hydraulic systems is essential for reducing unplanned downtime in manufacturing equipment such as presses, injection units and hydraulic power packs. This project focuses on multi-sensor, multi-target condition monitoring using a hydraulic test-rig dataset that contains 2,205 working cycles of 60 seconds each. Every cycle is recorded by 17 process sensors: six pressure sensors (PS1–PS6), a motor power sensor (EPS1), two volume-flow sensors (FS1, FS2), four temperature sensors (TS1–TS4), a vibration sensor (VS1), and three virtual sensors for cooling efficiency (CE), cooling power (CP) and system efficiency factor (SE). The condition of four components – the cooler, valve, pump and hydraulic accumulator – is labelled separately for every cycle.

The proposed system first loads the target labels and performs exploratory data analysis, showing the distribution of each component condition, a sample pressure and flow cycle, and how the cooling efficiency and motor power change with cooler degradation and pump leakage. Seven statistical features (mean, median, standard deviation, minimum, maximum, range and average rate of change) are then extracted from every sensor in every cycle, converting the raw signals into a feature matrix of 2,205 cycles × 119 features. The features are standardised using StandardScaler and the data is divided into 70% training (1,543 cycles) and 30% testing (662 cycles) sets.

Classical machine learning models, namely Random Forest, Support Vector Machine (RBF kernel) and Gradient Boosting, are built to classify the condition of the cooler. A single-target Random Forest is then compared with a multi-output Random Forest that predicts all four components together. The Random Forest achieves 100% accuracy for the cooler, 94.71% for the valve, 99.40% for the pump and 97.28% for the hydraulic accumulator. The models are analysed using confusion matrices, ROC-AUC curves and Random Forest feature importance, which identify the most influential sensors for each component. Based on these findings, component-wise preventive maintenance recommendations are proposed to reduce press downtime and avoid secondary damage.

Keywords: Hydraulic System, Predictive Maintenance, Condition Monitoring, Multi-Sensor Data, Feature Engineering, Random Forest, Support Vector Machine, Gradient Boosting, Multi-Output Classification, ROC-AUC, Feature Importance.

CODING

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import glob
import warnings
warnings.filterwarnings('ignore')

# Set plot style
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette('muted')

# Load profile.csv which contains target labels
# Columns: 1: Cooler condition, 2: Valve condition, 3: Internal pump
# leakage, 4: Hydraulic accumulator, 5: stable flag
profile_path = 'Datasets/profile.csv'
profile = pd.read_csv(profile_path, sep='\t', header=None)
profile.columns = ['Cooler_Condition', 'Valve_Condition', 'Pump_Leakage',
                   'Hydraulic_Accumulator', 'Stable_Flag']
print(f"Profile shape: {profile.shape}")
display(profile.head())
```

DATA LOADING AND TARGET LABELS OUTPUT

Profile shape: (2205, 5)

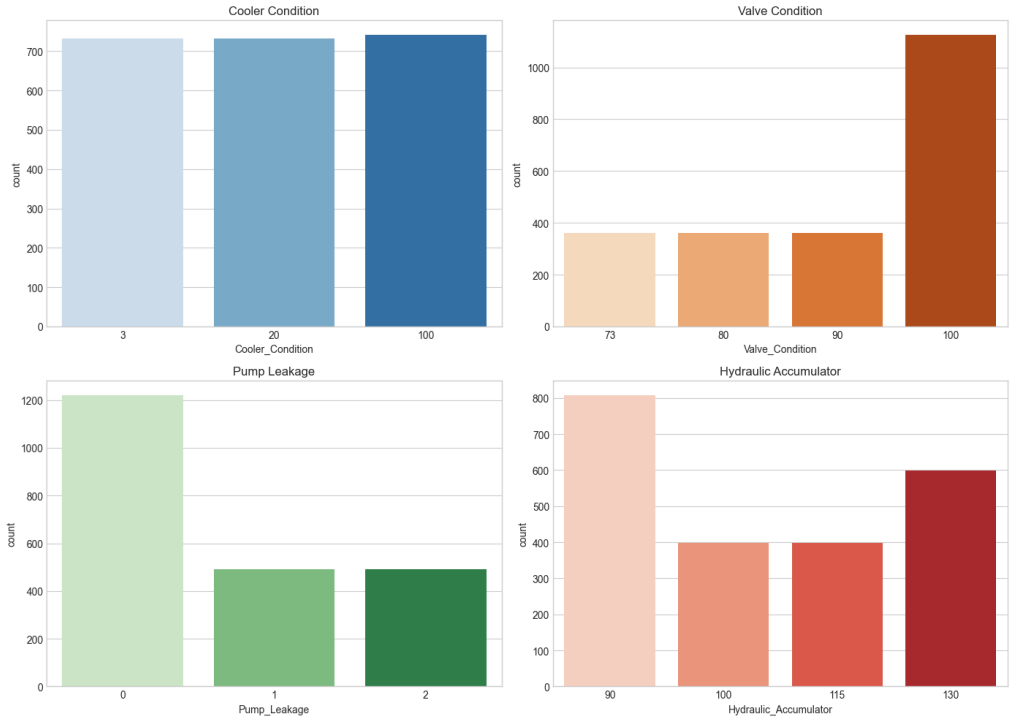
	Cooler_Condition	Valve_Condition	Pump_Leakage	Hydraulic_Accumulator	Stable_Flag
0	3	100	0	130	1
1	3	100	0	130	1
2	3	100	0	130	1
3	3	100	0	130	1
4	3	100	0	130	1

The table shows the first five rows of the target label file (profile.csv). The file contains 2,205 rows, one for each 60-second working cycle, and five columns: the condition of the cooler, valve, pump and hydraulic accumulator, and a flag indicating whether the cycle was stable. Each component has its own set of condition levels: the cooler takes the values 3, 20 and 100 (percentage of cooling efficiency, 100 being a fully working cooler), the valve takes 73, 80, 90 and 100 (percentage of switching behaviour, 100 being optimal), the pump

leakage takes 0, 1 and 2 (none, weak and severe leakage) and the accumulator takes 90, 100, 115 and 130 (bar of pre-charge pressure, 130 being optimal).

```
# Plot Target Distributions
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
sns.countplot(x='Cooler_Condition', data=profile, ax=axes[0,0],
        palette='Blues').set_title('Cooler Condition')
sns.countplot(x='Valve_Condition', data=profile, ax=axes[0,1],
        palette='Oranges').set_title('Valve Condition')
sns.countplot(x='Pump_Leakage', data=profile, ax=axes[1,0],
        palette='Greens').set_title('Pump Leakage')
sns.countplot(x='Hydraulic_Accumulator', data=profile, ax=axes[1,1],
        palette='Reds').set_title('Hydraulic Accumulator')
plt.tight_layout()
plt.show()
```

TARGET DISTRIBUTION OUTPUT



The count plots show how many cycles belong to each condition level of the four components. The cooler (3, 20, 100) and the accumulator levels are reasonably balanced, with the cooler having about 730 cycles per level. The valve condition is imbalanced, with the optimal level (100) having roughly 1,125 cycles compared with about 365 cycles for each degraded level. The pump leakage is also imbalanced, with about 1,220 cycles having no leakage and about 490 cycles each for weak and severe leakage. This imbalance is one reason why per-class metrics such as the confusion matrix and ROC-AUC are examined in addition to accuracy.

```
# Sensor files dictionary mapped to their sampling rates
sensor_files = {
    'PS1': 100, 'PS2': 100, 'PS3': 100, 'PS4': 100, 'PS5': 100, 'PS6': 100,
    'EPS1': 100,
    'FS1': 10, 'FS2': 10,
    'TS1': 1, 'TS2': 1, 'TS3': 1, 'TS4': 1,
    'VS1': 1, 'CE': 1, 'CP': 1, 'SE': 1
}

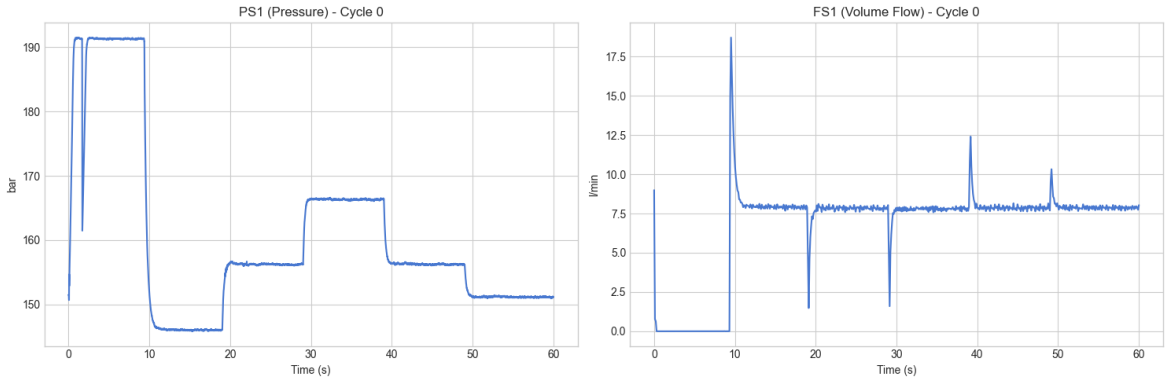
# Plot cycle 0 for PS1 (Pressure) and FS1 (Volume Flow)
plt.figure(figsize=(15, 5))

# Plot PS1 (100 Hz -> 6000 points per cycle)
ps1 = pd.read_csv('Datasets/PS1.csv', sep='\t', header=None)
plt.subplot(1, 2, 1)
plt.plot(np.linspace(0, 60, 6000), ps1.iloc[0, :])
plt.title('PS1 (Pressure) - Cycle 0')
plt.xlabel('Time (s)')
plt.ylabel('bar')

# Plot FS1 (10 Hz -> 600 points per cycle)
fs1 = pd.read_csv('Datasets/FS1.csv', sep='\t', header=None)
plt.subplot(1, 2, 2)
plt.plot(np.linspace(0, 60, 600), fs1.iloc[0, :])
plt.title('FS1 (Volume Flow) - Cycle 0')
plt.xlabel('Time (s)')
plt.ylabel('l/min')

plt.tight_layout()
plt.show()
```

SENSOR CYCLE STRUCTURE OUTPUT



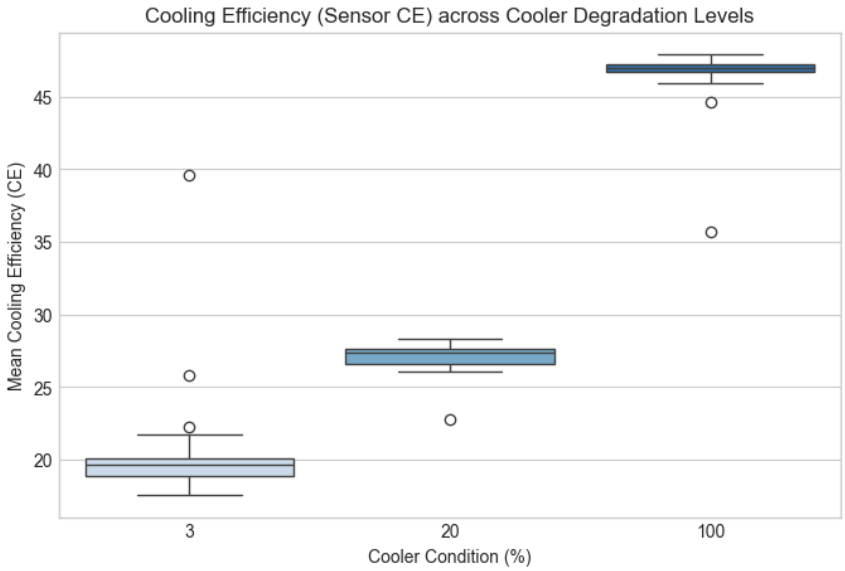
Each sensor has its own file in which every row is one 60-second cycle. The pressure sensors (PS1–PS6) and the motor power sensor (EPS1) are sampled at 100 Hz (6,000 values per cycle), the volume-flow sensors (FS1, FS2) at 10 Hz (600 values per cycle), and the temperature sensors (TS1–TS4), vibration sensor (VS1) and the virtual sensors (CE, CP, SE) at 1 Hz (60 values per cycle). The plot of cycle 0 shows that the load cycle is repeatable: the

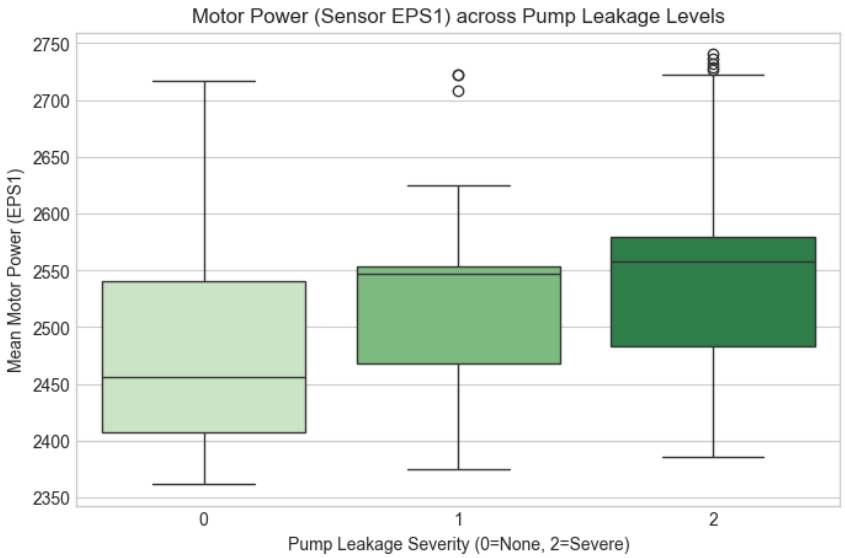
pressure PS1 rises to about 190 bar, falls to about 146 bar and then moves through several plateaus between 150 and 167 bar, while the flow FS1 shows sharp transients whenever the valve switches. These repeating shapes are what the component faults distort.

```
# EDA: Sensor signal across degradation levels
# Cooling Efficiency (CE) vs Cooler Condition
ce_sensor = pd.read_csv('Datasets/CE.csv', sep='\t', header=None)
temp_df = pd.DataFrame({
   'Mean_Cooling_Efficiency': ce_sensor.mean(axis=1),
   'Cooler_Condition': profile['Cooler_Condition']
})
plt.figure(figsize=(8, 5))
sns.boxplot(x='Cooler_Condition', y='Mean_Cooling_Efficiency',
        data=temp_df, palette='Blues')
plt.title('Cooling Efficiency (Sensor CE) across Cooler Degradation Levels')
plt.xlabel('Cooler Condition (%)')
plt.ylabel('Mean Cooling Efficiency (CE)')
plt.show()
```

```
# Motor Power (EPS1) vs Pump Leakage
eps1_sensor = pd.read_csv('Datasets/EPS1.csv', sep='\t', header=None)
temp_df2 = pd.DataFrame({
   'Mean_Motor_Power': eps1_sensor.mean(axis=1),
   'Pump_Leakage': profile['Pump_Leakage']
})
plt.figure(figsize=(8, 5))
sns.boxplot(x='Pump_Leakage', y='Mean_Motor_Power',
        data=temp_df2, palette='Greens')
plt.title('Motor Power (Sensor EPS1) across Pump Leakage Levels')
plt.xlabel('Pump Leakage Severity (0=None, 2=Severe)')
plt.ylabel('Mean Motor Power (EPS1)')
plt.show()
```

EDA ACROSS DEGRADATION LEVELS OUTPUT





The first box plot shows that the mean cooling efficiency separates the three cooler conditions almost perfectly: it is about 19–20 for the near-failed cooler (3%), about 27 for the reduced cooler (20%) and about 47 for the fully working cooler (100%), with only a few outliers. This explains why the cooler is the easiest component to classify. The second box plot shows that the mean motor power rises with pump leakage (median of about 2,455 for no leakage, 2,548 for weak leakage and 2,558 for severe leakage), because the motor works harder to compensate for the leakage. However, the boxes overlap considerably, so a single sensor mean is not enough to separate the leakage levels, which motivates the richer feature set built in the next step.

```
# Feature Engineering: compute per-cycle statistical features
def extract_features(sensor_name):
    filepath = f'Datasets/{sensor_name}.csv'
    df = pd.read_csv(filepath, sep='\t', header=None)
```

```
    # Calculate cycle-wise features
    features = pd.DataFrame()
    features[f'{sensor_name}_mean'] = df.mean(axis=1)
    features[f'{sensor_name}_median'] = df.median(axis=1)
    features[f'{sensor_name}_std'] = df.std(axis=1)
    features[f'{sensor_name}_min'] = df.min(axis=1)
    features[f'{sensor_name}_max'] = df.max(axis=1)
    features[f'{sensor_name}_range'] = df.max(axis=1) - df.min(axis=1)
    features[f'{sensor_name}_rate_of_change'] = df.diff(axis=1).mean(axis=1)
    return features

all_features = []
for sensor in sensor_files.keys():
    print(f"Processing {sensor} ...")
    sensor_features = extract_features(sensor)
    all_features.append(sensor_features)

# Combine all features into a single dataframe
X = pd.concat(all_features, axis=1)
print(f"\nExtracted features shape: {X.shape}")
display(X.head())
```

FEATURE ENGINEERING OUTPUT

```
Processing PS1 ...
Processing PS2 ...
Processing PS3 ...
Processing PS4 ...
Processing PS5 ...
Processing PS6 ...
Processing EPS1 ...
Processing FS1 ...
Processing FS2 ...
Processing TS1 ...
Processing TS2 ...
Processing TS3 ...
Processing TS4 ...
Processing VS1 ...
Processing CE ...
Processing CP ...
Processing SE ...

Extracted features shape: (2205, 119)
```

	PS1_mean	PS1_median	PS1_std	PS1_min	PS1_max	PS1_range	PS1_rate_of_change
0	160.673492	156.25	13.939309	145.83	191.51	45.68	-0.000047
1	160.603320	156.06	14.118967	145.73	191.47	45.74	-0.000042

	PS1_mean	PS1_median	PS1_std	PS1_min	PS1_max	PS1_range	PS1_rate_of_change
2	160.347720	155.72	14.192619	145.37	191.41	46.04	-0.000032
3	160.188088	155.56	14.227803	145.14	191.34	46.20	-0.000033
4	160.000472	155.34	14.276434	144.95	191.41	46.46	-0.000037

Every raw sensor signal is reduced to seven statistical features per cycle: mean, median, standard deviation, minimum, maximum, range (maximum minus minimum) and the average rate of change (the mean of the first differences). With 17 sensors, this gives 17 × 7 = 119 features for each of the 2,205 cycles, so the feature matrix has the shape (2,205, 119). The table shows the first five rows of the PS1 features as a sample. This step compresses up to 6,000 raw readings per sensor into seven numbers while preserving the level, spread and trend of the signal, which are exactly the characteristics that change when a component degrades.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
display(X_scaled.head())

# Map target variables
y_cooler = profile['Cooler_Condition']
y_valve = profile['Valve_Condition']
y_pump = profile['Pump_Leakage']
y_accumulator = profile['Hydraulic_Accumulator']
print("Targets ready for modeling.")

from sklearn.model_selection import train_test_split

# Create train/test splits for each target
X_train, X_test, y_cooler_train, y_cooler_test = train_test_split(
    X_scaled, y_cooler, test_size=0.3, random_state=42)
_, _, y_valve_train, y_valve_test = train_test_split(
    X_scaled, y_valve, test_size=0.3, random_state=42)
_, _, y_pump_train, y_pump_test = train_test_split(
    X_scaled, y_pump, test_size=0.3, random_state=42)
_, _, y_acc_train, y_acc_test = train_test_split(
    X_scaled, y_accumulator, test_size=0.3, random_state=42)

print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")
```

NORMALIZATION AND TRAIN-TEST SPLIT OUTPUT

```
Targets ready for modeling.
Train shape: (1543, 119), Test shape: (662, 119)
```

	PS1_mean	PS1_median	PS1_std	PS1_min	PS1_max	PS1_range	PS1_rate_of_change
0	0.040052	-0.128150	-0.656258	1.073368	1.004103	-1.072761	-0.059970
1	0.025116	-0.145978	-0.569368	1.027831	0.937749	-1.036019	-0.053513
2	-0.029286	-0.177882	-0.533747	0.863898	0.838219	-0.852309	-0.040601
3	-0.063262	-0.192895	-0.516730	0.759163	0.722101	-0.754330	-0.042753
4	-0.103194	-0.213538	-0.493210	0.672643	0.838219	-0.595115	-0.047057

The 119 features lie on very different scales (for example pressures in bar, motor power in watts and temperatures in °C), so they are standardised to zero mean and unit variance using StandardScaler. The table shows that the PS1 features now fall around zero. The four condition columns are then separated as individual target variables. The data is split into 70% training and 30% testing sets with a fixed random seed, giving 1,543 training cycles and 662 test cycles. The same random seed is used for every target, so all four components share exactly the same train and test cycles.

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Example: Training Models for Cooler Condition
rf_cooler = RandomForestClassifier(n_estimators=100, random_state=42)
svm_cooler = SVC(kernel='rbf', probability=True, random_state=42)
gb_cooler = GradientBoostingClassifier(n_estimators=100, random_state=42)

# Train models
print("Training Random Forest ...")
rf_cooler.fit(X_train, y_cooler_train)
print("Training SVM ...")
svm_cooler.fit(X_train, y_cooler_train)
print("Training Gradient Boosting ...")
gb_cooler.fit(X_train, y_cooler_train)

# Evaluate Random Forest
rf_pred = rf_cooler.predict(X_test)
print("\nRandom Forest - Cooler Condition Accuracy:",
    accuracy_score(y_cooler_test, rf_pred))
print("\nClassification Report:\n",
    classification_report(y_cooler_test, rf_pred))

# Confusion Matrix for RF
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_cooler_test, rf_pred),
        annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix: RF - Cooler Condition")
plt.ylabel('Actual')
```

```
plt.xlabel('Predicted')
plt.show()
```

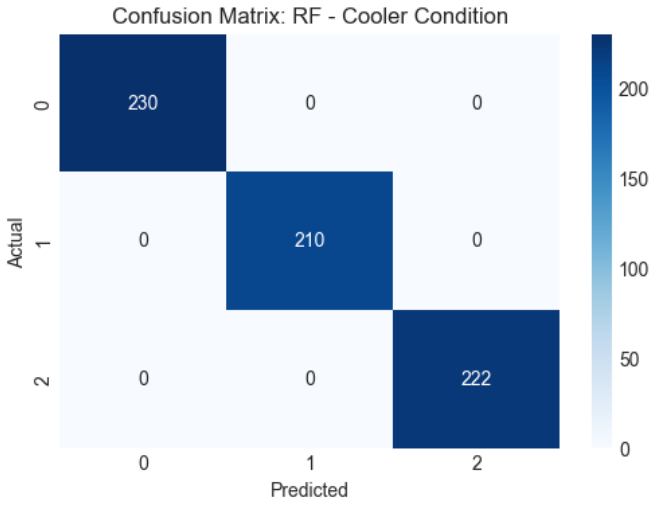
SINGLE-TARGET MODEL (COOLER CONDITION) OUTPUT

```
Training Random Forest ...
Training SVM ...
Training Gradient Boosting ...
```

Random Forest - Cooler Condition Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
3	1.00	1.00	1.00	230
20	1.00	1.00	1.00	210
100	1.00	1.00	1.00	222
accuracy			1.00	662
macro avg	1.00	1.00	1.00	662
weighted avg	1.00	1.00	1.00	662



Three classical machine learning algorithms – Random Forest (100 trees), Support Vector Machine with an RBF kernel and Gradient Boosting (100 estimators) – are trained on the 1,543 training cycles to classify the cooler condition. The Random Forest is evaluated on the 662 test cycles and reaches an accuracy of 100%: the confusion matrix is perfectly diagonal, with all 230, 210 and 222 cycles of the three cooler levels classified correctly and every precision, recall and F1-score equal to 1.00. This is consistent with the EDA, where the cooling-efficiency signal separates the three cooler levels almost completely.

```
from sklearn.multioutput import MultiOutputClassifier

# Prepare combined target matrix
y_multi = pd.concat([y_cooler, y_valve, y_pump, y_accumulator], axis=1)
y_multi_train, y_multi_test = train_test_split(
    y_multi, test_size=0.3, random_state=42)

# Initialize Multi-Output Random Forest
rf_multi = MultiOutputClassifier(
    RandomForestClassifier(n_estimators=100, random_state=42))

print("Training Multi-Output Random Forest...")
rf_multi.fit(X_train, y_multi_train)
y_multi_pred = rf_multi.predict(X_test)

# Evaluate
print("Multi-Output Model trained. Evaluating per target:")
for i, col in enumerate(y_multi.columns):
    acc = accuracy_score(y_multi_test.iloc[:, i], y_multi_pred[:, i])
    print(f"{col} Accuracy: {acc:.4f}")
```

MULTI-OUTPUT CLASSIFICATION OUTPUT

```
Training Multi-Output Random Forest...
Multi-Output Model trained. Evaluating per target:
Cooler_Condition Accuracy: 1.0000
Valve_Condition Accuracy: 0.9471
Pump_Leakage Accuracy: 0.9940
Hydraulic_Accumulator Accuracy: 0.9728
```

Component	Condition levels	Test accuracy (Multi-output RF)
Cooler	3, 20, 100	100.00%
Valve	73, 80, 90, 100	94.71%
Pump leakage	0, 1, 2	99.40%
Hydraulic accumulator	90, 100, 115, 130	97.28%

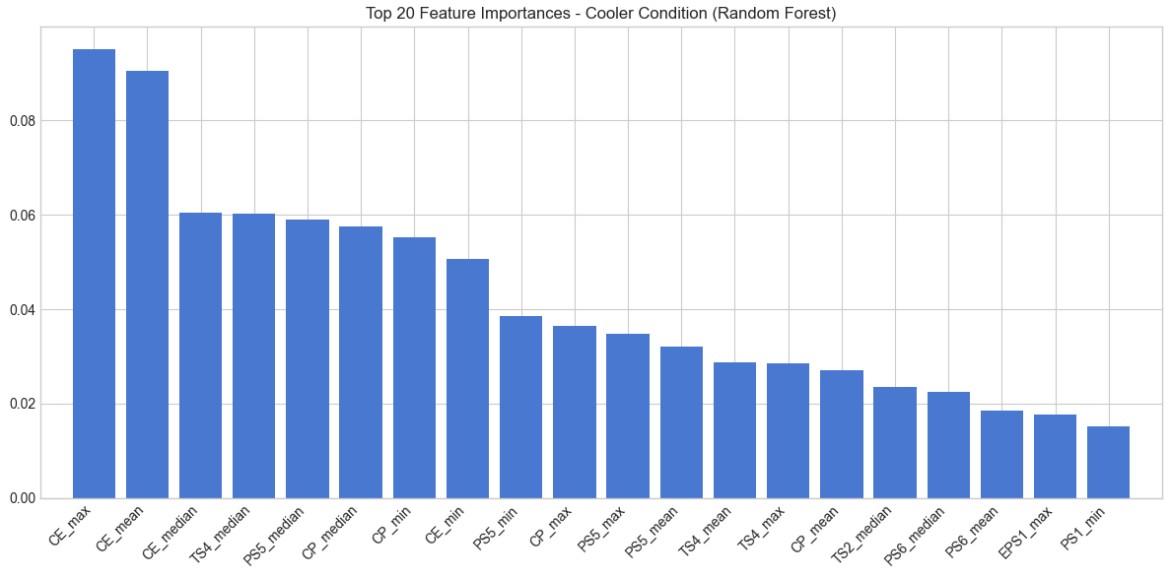
Because the four components degrade independently, the problem is a multi-target one. A MultiOutputClassifier wraps the Random Forest and trains one forest per component on the same 119 features, so a single call predicts the condition of all four components for a cycle. The cooler is classified perfectly, the pump leakage reaches 99.40%, the accumulator 97.28% and the valve is the hardest component with 94.71%. The multi-output model fits one independent forest per target with the same settings and random seed as the single-target forests, so its accuracies are identical to those of the single-target models shown later for the valve, pump and accumulator. The advantage of the multi-output approach is therefore

practical (one object, one training call and one prediction for the whole machine) rather than a gain in accuracy.

```
# Feature Importance from the Cooler Random Forest Model
importances = rf_cooler.feature_importances_
indices = np.argsort(importances)[::-1]

# Plot top 20 features
plt.figure(figsize=(12, 6))
plt.title("Top 20 Feature Importances - Cooler Condition (Random Forest)")
plt.bar(range(20), importances[indices][:20], align="center")
plt.xticks(range(20), [X.columns[i] for i in indices[:20]],
        rotation=45, ha='right')
plt.xlim([-1, 20])
plt.tight_layout()
plt.show()
```

COOLER FEATURE IMPORTANCE OUTPUT



The bar chart shows the 20 most important features of the cooler Random Forest. The two most influential features are CE_max and CE_mean (importance of about 0.095 and 0.090), followed by CE_median, TS4_median, PS5_median, CP_median, CP_min and CE_min (about 0.05–0.06 each). The cooling-efficiency (CE) and cooling-power (CP) virtual sensors, together with the temperature sensor TS4 and the pressure sensor PS5, therefore carry most of the information about the cooler. This is physically sensible, since a degraded cooler directly lowers the heat removed from the oil.

```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
from itertools import cycle

# Binarize the output for multi-class ROC AUC
```

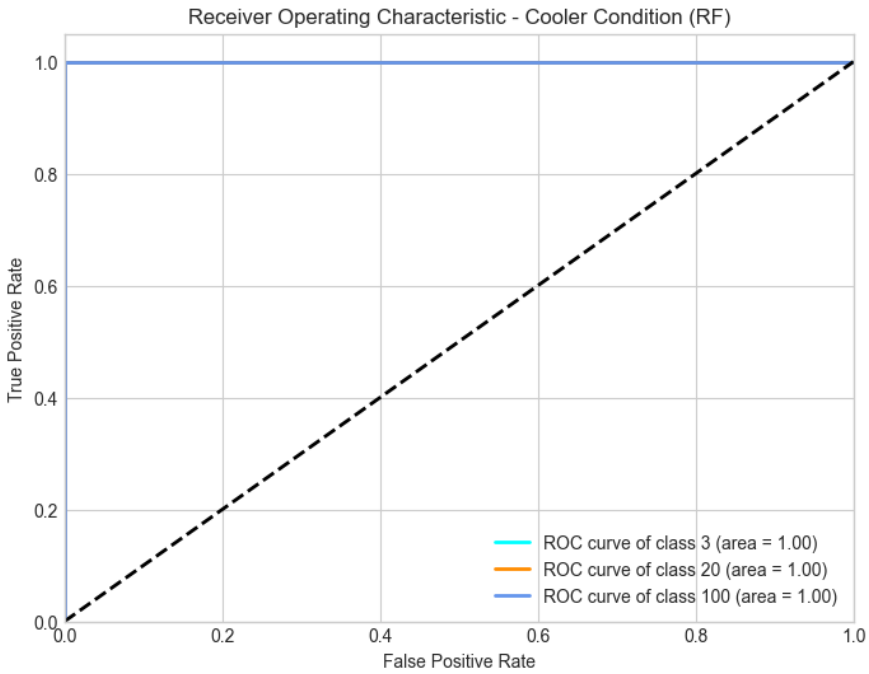
```
classes = np.unique(y_cooler_test)
y_test_bin = label_binarize(y_cooler_test, classes=classes)
n_classes = y_test_bin.shape[1]

# Predict probabilities
y_score = rf_cooler.predict_proba(X_test)

# Compute ROC curve and ROC area for each class
fpr = dict(); tpr = dict(); roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot all ROC curves
plt.figure(figsize=(8, 6))
colors = cycle(['aqua', 'darkorange', 'cornflowerblue'])
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label='ROC curve of class {0} (area = {1:0.2f})'
                 ''.format(classes[i], roc_auc[i]))
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic - Cooler Condition (RF)')
plt.legend(loc="lower right")
plt.show()
```

COOLER ROC-AUC OUTPUT



The ROC curves are drawn using a one-vs-rest approach, in which each cooler level is treated as the positive class against the other two. All three curves (classes 3, 20 and 100) coincide with the top-left corner of the plot, giving an area under the curve (AUC) of 1.00 for every class. This means the Random Forest ranks every test cycle of a class above all cycles of the other classes, confirming the perfect separation seen in the confusion matrix.

```
# Training and Evaluation for Valve, Pump, and Accumulator
remaining_targets = {
    'Valve_Condition': y_valve_train,
    'Pump_Leakage': y_pump_train,
    'Hydraulic_Accumulator': y_acc_train
}
remaining_test = {
    'Valve_Condition': y_valve_test,
    'Pump_Leakage': y_pump_test,
    'Hydraulic_Accumulator': y_acc_test
}

for target_name in remaining_targets.keys():
    print(f"\n{'='*50}")
    print(f"Processing target: {target_name}")
    print(f"{'='*50}")

    y_tr = remaining_targets[target_name]
    y_te = remaining_test[target_name]

    # Train Random Forest
    rf = RandomForestClassifier(n_estimators=100, random_state=42)
    rf.fit(X_train, y_tr)

    # Predict
    y_pred = rf.predict(X_test)
    y_score = rf.predict_proba(X_test)

    # 1. Confusion Matrix
    plt.figure(figsize=(6,4))
    sns.heatmap(confusion_matrix(y_te, y_pred), annot=True, fmt='d',
                cmap='Blues')
    plt.title(f"Confusion Matrix: {target_name}")
    plt.ylabel('Actual')
    plt.xlabel('Predicted')
    plt.show()

    # 2. Feature Importance
    importances = rf.feature_importances_
    indices = np.argsort(importances)[::-1]
    plt.figure(figsize=(10, 5))
    plt.title(f"Top 20 Feature Importances: {target_name}")
    plt.bar(range(20), importances[indices][:20], align="center")
    plt.xticks(range(20), [X.columns[i] for i in indices[:20]],
               rotation=45, ha='right')
    plt.xlim([-1, 20])
```

```
plt.tight_layout()
plt.show()

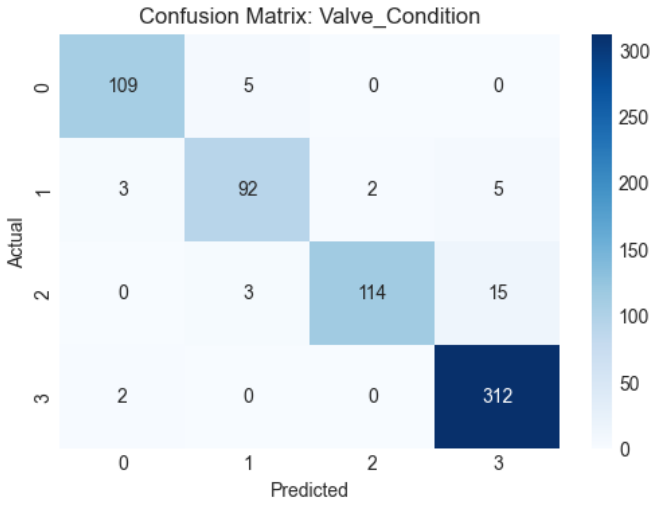
# 3. ROC-AUC
classes = np.unique(y_te)
y_test_bin = label_binarize(y_te, classes=classes)
n_classes = y_test_bin.shape[1]
if n_classes == 1:
    n_classes = 2
    y_test_bin = np.hstack((1 - y_test_bin, y_test_bin))

fpr = dict(); tpr = dict(); roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure(figsize=(8, 5))
colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'red'])
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label=f'ROC curve of class {classes[i]} '
                   f'(area = {roc_auc[i]:0.2f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'Receiver Operating Characteristic: {target_name}')
plt.legend(loc="lower right")
plt.show()
```

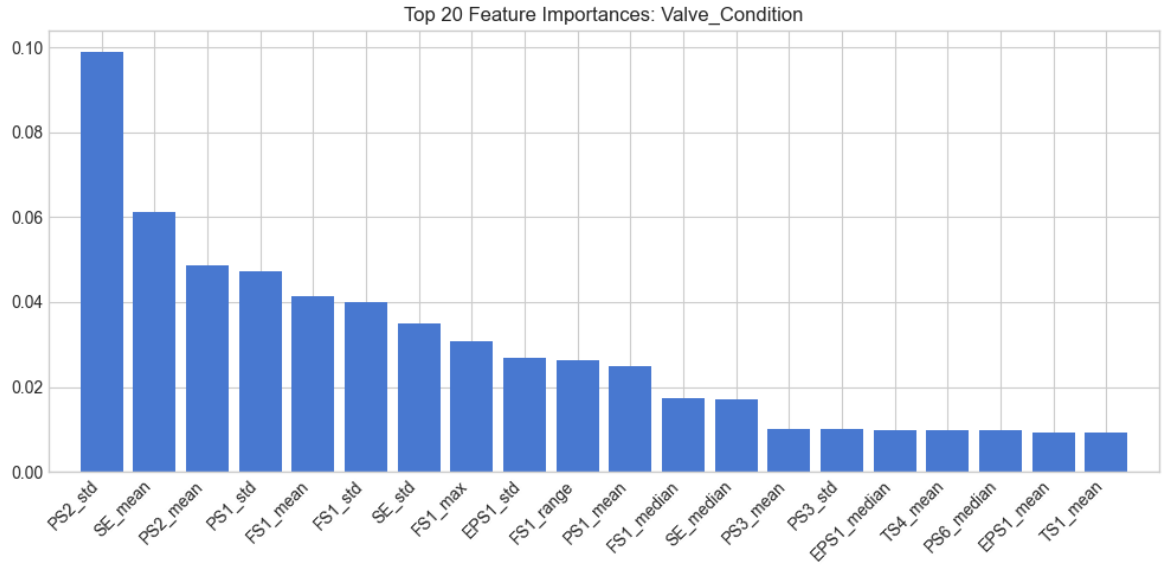
VALVE CONDITION OUTPUT

CONFUSION MATRIX



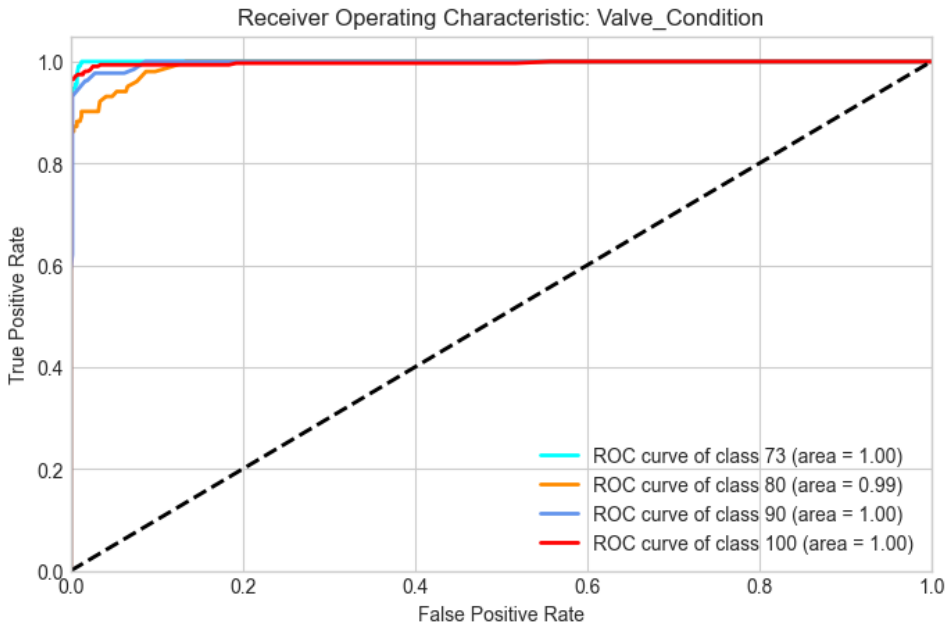
In the confusion matrix, classes 0 to 3 correspond to the valve levels 73, 80, 90 and 100 in increasing order. The model classifies 109 of 114 cycles of level 73, 92 of 102 cycles of level 80, 114 of 132 cycles of level 90 and 312 of 314 cycles of the optimal level 100 correctly, giving an overall accuracy of 94.71% (627 of 662 cycles). Almost all errors occur between neighbouring degradation levels; for example, 15 cycles of level 90 are predicted as the optimal level 100, which is the most common mistake. The optimal valve is detected with the highest recall (about 99.4%), while level 90 has the lowest recall (about 86.4%).

FEATURE IMPORTANCE



The valve is characterised mainly by pressure and flow dynamics. The most important feature is PS2_std (about 0.099), followed by SE_mean (about 0.061), PS2_mean, PS1_std, FS1_mean, FS1_std and SE_std. The standard deviation of the pressure signals is important because a lagging valve changes how sharply the pressure steps in the load cycle occur, while the flow sensor FS1 reflects the switching transients seen in the sample cycle.

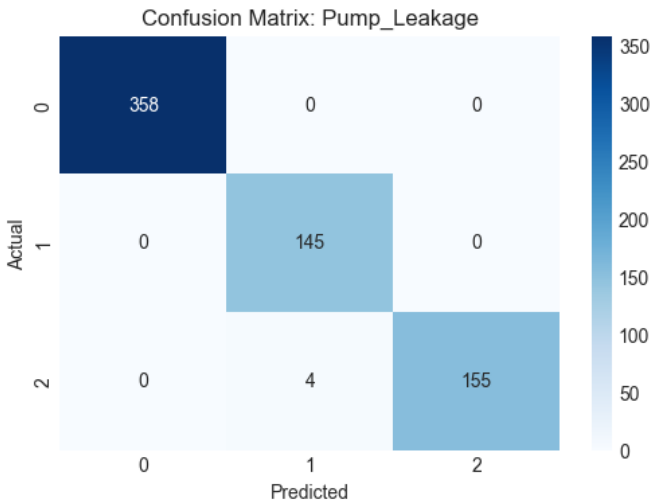
ROC-AUC ANALYSIS



The ROC curves of the four valve levels have an AUC of 1.00 for levels 73, 90 and 100 and 0.99 for level 80. Level 80 shows the slowest rise of the curve, which means that its probability scores overlap slightly with those of the neighbouring levels, in agreement with the confusion matrix.

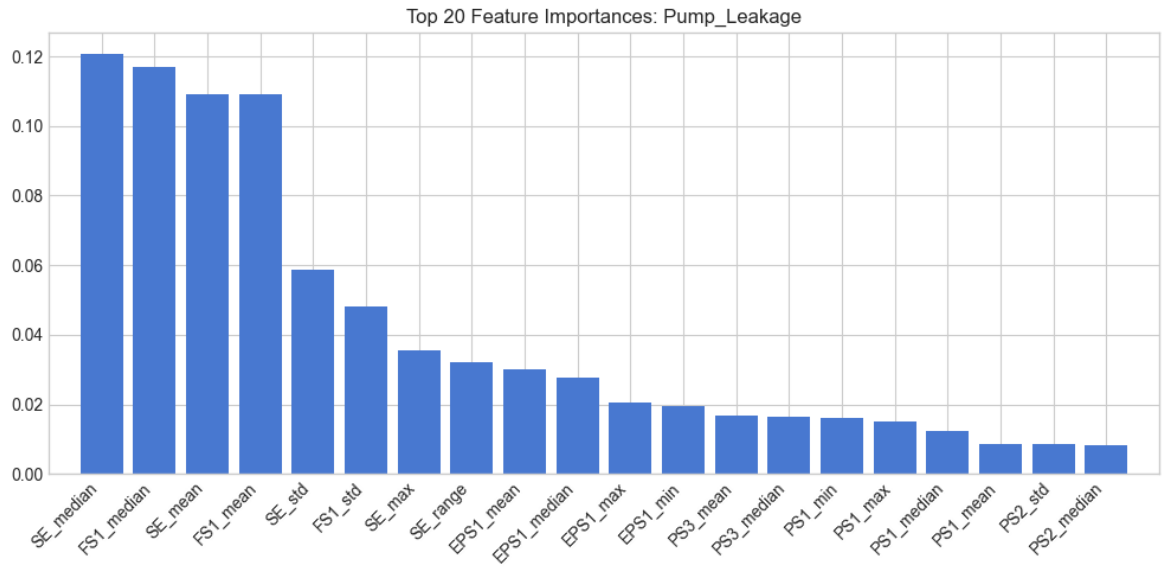
PUMP LEAKAGE OUTPUT

CONFUSION MATRIX



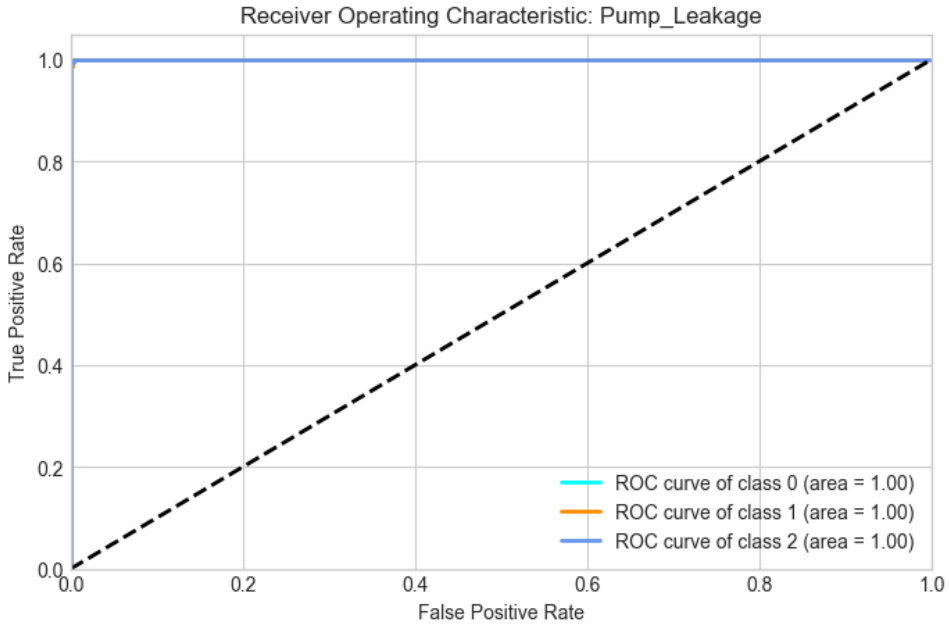
The pump leakage is classified almost perfectly, with an accuracy of 99.40% (658 of 662 cycles). All 358 cycles with no leakage and all 145 cycles with weak leakage are correct, and 155 of 159 cycles with severe leakage are correct. The only errors are 4 cycles of severe leakage predicted as weak leakage, which are neighbouring levels.

FEATURE IMPORTANCE



The most important features are SE_median, FS1_median, SE_mean and FS1_mean (about 0.11–0.12 each), followed by SE_std, FS1_std, SE_max and SE_range. The system efficiency factor (SE) and the volume flow (FS1) are the strongest indicators, because an internal leak lowers the useful flow delivered by the pump and thereby lowers the overall efficiency, while the motor power (EPS1) appears further down the list.

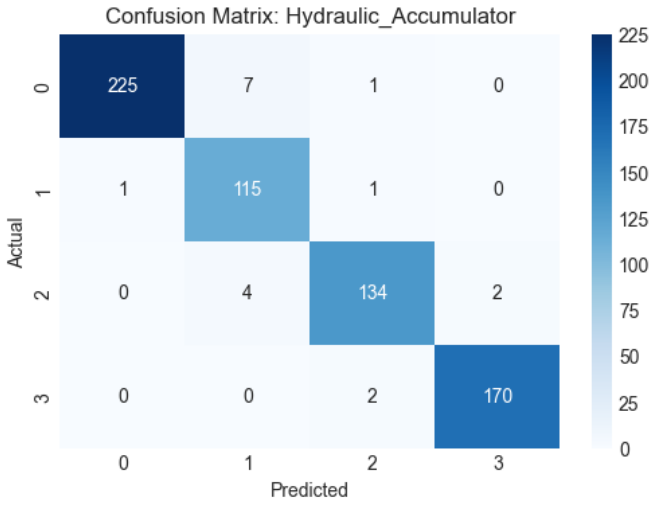
ROC-AUC ANALYSIS



All three ROC curves (no leakage, weak leakage and severe leakage) reach an AUC of 1.00, showing a very clean separation of the three leakage levels.

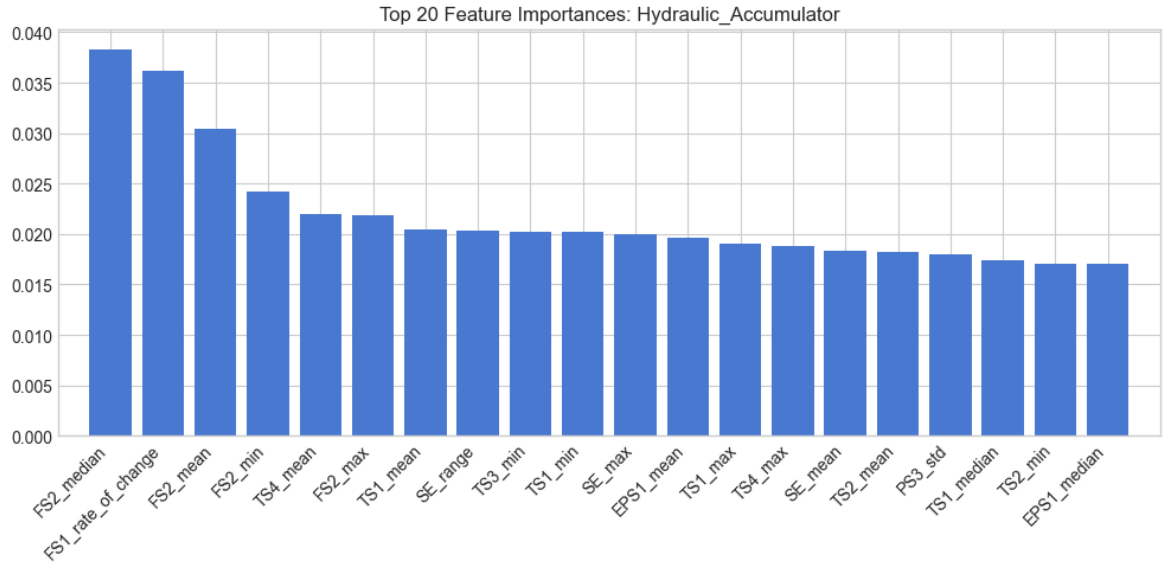
HYDRAULIC ACCUMULATOR OUTPUT

CONFUSION MATRIX



The four accumulator levels (90, 100, 115 and 130 bar, shown as classes 0 to 3) are classified with an accuracy of 97.28% (644 of 662 cycles). The recalls are about 96.6% for level 90 (225 of 233), 98.3% for level 100 (115 of 117), 95.7% for level 115 (134 of 140) and 98.8% for level 130 (170 of 172). The errors again occur between neighbouring pre-charge levels, mostly between level 90 and level 100 (7 cycles) and between level 100 and level 115 (4 cycles).

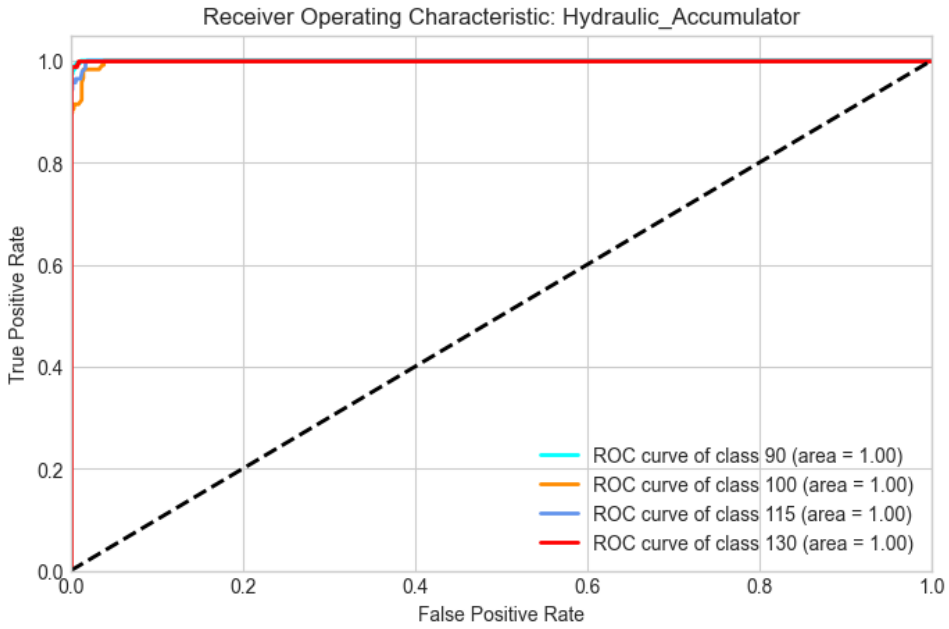
FEATURE IMPORTANCE



The importance is spread over many features rather than concentrated in a few, and the highest value is only about 0.038. The top features are FS2_median, FS1_rate_of_change, FS2_mean, FS2_min and TS4_mean, followed by several temperature and flow features.

The accumulator affects how the pressure and flow settle after each valve switching, so the flow sensors FS1 and FS2 and their rate of change are the most informative.

ROC-AUC ANALYSIS



The ROC curves of all four accumulator levels (90, 100, 115 and 130) have an AUC of 1.00, with the level 100 curve rising marginally later than the others, which agrees with its few misclassifications.

SINGLE-TARGET VS MULTI-OUTPUT COMPARISON

Component	Single-target RF accuracy	Multi-output RF accuracy	Lowest class recall	Mean AUC
Cooler	100.00%	100.00%	100.0% (all levels)	1.00
Valve	94.71%	94.71%	86.4% (level 90)	0.99–1.00
Pump leakage	99.40%	99.40%	97.5% (severe)	1.00
Hydraulic accumulator	97.28%	97.28%	95.7% (level 115)	1.00

The table compares the single-target Random Forest of each component with the multi-output Random Forest. The accuracies are identical because both approaches fit an independent forest with the same settings for each component on the same training cycles. The cooler is the easiest component, followed by the pump, the accumulator and finally the valve. The valve is the most difficult component because its three degraded levels differ only slightly in the pressure and flow dynamics and because the optimal level dominates the data (about 51% of the cycles).

COMPARISON OF THE THREE CLASSICAL ALGORITHMS (COOLER CONDITION)

Algorithm	Configuration	Cooler test accuracy
Random Forest	100 trees, random_state = 42	100.00%
Support Vector Machine	RBF kernel, probability = True	[fill from notebook output]
Gradient Boosting	100 estimators, random_state = 42	[fill from notebook output]

Note: all three algorithms are trained in the notebook, but only the Random Forest accuracy is printed in the output shown above. Add the line accuracy_score(y_cooler_test, svm_cooler.predict(X_test)) (and the same for gb_cooler) to the notebook and fill in the two values in this table.

COMPONENT-WISE MAINTENANCE RECOMMENDATIONS

The results are converted into a condition-based preventive maintenance plan. Instead of replacing parts at fixed intervals, every working cycle is passed through the trained classifiers and the predicted condition level of each component triggers an action.

Component	Key sensors to monitor	Recommended maintenance action
Cooler	CE, CP, TS4, PS5	Inspect and clean the cooler when the predicted level drops to 20%. Replace it before it reaches 3%, since oil overheating accelerates ageing of the other components.
Valve	PS2, PS1, SE, FS1	Schedule an inspection at level 90 and a replacement at level 80, because the valve classes are the least separable and a safety margin is needed. Level 73 is treated as an immediate stop.
Pump	SE, FS1, EPS1	Continue normal operation for no leakage, plan a seal or pump service at weak leakage and replace or overhaul the pump at severe leakage, before the motor is overloaded.
Hydraulic accumulator	FS2, FS1 rate of change, TS4	Re-charge the pre-charge pressure when the predicted level drops from 130 to 115 and replace the bladder or accumulator at level 90 to avoid pressure shocks.

EXPECTED BUSINESS IMPACT

Reduced press downtime: Early, component-specific warnings allow repairs to be planned in scheduled stops rather than after a breakdown of the press.

Avoided secondary damage: Replacing a degraded cooler, pump or accumulator in time protects the neighbouring parts from overheating, overload and pressure shocks.

Lower maintenance cost: Parts are replaced based on their actual condition instead of fixed intervals, which avoids both premature replacement and late detection.

Simple deployment: A single multi-output model gives the status of all four components from one set of sensor features, which keeps deployment and monitoring simple.

The monetary saving depends on the cost of an hour of press downtime and on the price of each component in the plant, so these plant-specific figures should be inserted to turn the above benefits into a cost estimate.

CONCLUSION

This project built a multi-sensor condition-monitoring system for a hydraulic test rig. Per-cycle statistical features (mean, median, standard deviation, minimum, maximum, range and rate of change) were extracted from 17 sensors, giving 119 features for 2,205 cycles. A Random Forest classifier identified the condition of the cooler with 100% accuracy, the pump leakage with 99.40%, the hydraulic accumulator with 97.28% and the valve with 94.71% on unseen test cycles, with ROC-AUC values of about 1.00 for nearly every class. The feature importance analysis showed which sensors matter for each component: the cooling efficiency and cooling power for the cooler, the pressure variation for the valve, the efficiency factor and flow for the pump, and the flow sensors for the accumulator.

The valve and accumulator remain the most difficult components, and their errors occur between neighbouring degradation levels. Possible future work includes tuning the hyperparameters, handling the class imbalance, fitting the scaler only on the training data, validating with cross-validation, and extending the work to a digital-twin style monitor that tracks the degradation of each component over time.